CSE 341 — Concepts of Programming Languages
Gebze Technical University | Spring 2026

# WorkoutLang

*A Domain-Specific Language for Workout Programming*

Design Specification — D1 (Part 1) | Due: 8 May 2026

**Implementation Language: C**

## 4.1 Language Overview

WorkoutLang is a domain-specific language for designing, structuring, and documenting resistance-training and cardio workout programs. Its target users are fitness coaches and athletes who want to describe workout sessions — exercises, sets, repetitions, rest periods, and progression rules — in a readable, unambiguous format without learning a general-purpose programming language.

**Evaluation Against Sebesta's Language Evaluation Criteria (Ch. 1):**

- **Readability (§1.3.1):** WorkoutLang prioritizes readability above all. Keywords such as set, reps, rest, load, and repeat directly mirror the spoken vocabulary of fitness coaching. A trainer can read a WorkoutLang program aloud and understand it without formal language training.
- **Writability (§1.3.2):** The language is narrow by design, offering only the constructs needed for workout programming. This intentional restriction improves writability for the target domain while sacrificing generality.
- **Reliability (§1.3.3):** Strong typing prevents category errors such as using a duration value where a repetition count is expected. The type system is knowingly prioritized over flexibility.
- **Cost:** WorkoutLang knowingly sacrifices computational generality and Turing-completeness. It is not a general-purpose language; its goal is expressive power within the fitness domain at minimal learning cost.

## 4.2 Lexical Structure

The following table lists all token categories recognized by the WorkoutLang lexer, along with their regular expressions or enumerated values.

| Token Category | Pattern / Enumerated Values | Examples |
|---|---|---|
| IDENT | [a-zA-Z_][a-zA-Z0-9_]* (not a keyword) | benchpress, myWeek, vol |
| INT_LIT | [0-9]+ | 3, 10, 90 |
| FLOAT_LIT | [0-9]+ "." [0-9]+ | 1.5, 72.5 |
| DURATION_LIT | [0-9]+ ("s" \| "min" \| "hr") | 90s, 2min, 1hr |
| WEIGHT_LIT | [0-9]+ ("kg" \| "lb") | 80kg, 135lb |
| STRING_LIT | "\"" [^\"\ n]* "\"" | "Deload next week" |
| BOOL_LIT | true \| false | true, false |
| KEYWORDS | workout \| routine \| day \| set \| reps \| rest \| repeat \| weeks \| load \| if \| else \| while \| return \| print \| int \| float \| bool \| duration \| weight | set, reps, if |

| | | |
|---|---|---|
| OPERATORS | + - * / == != < <= > >= && \|\| ! | >, ==, && |
| SEPARATORS | { } ( ) , : x | {, }, x |

## 4.3 Syntax (EBNF Grammar)

The complete grammar is given in Sebesta EBNF notation (§3.1–3.2). Terminal symbols appear in double quotes or ALL_CAPS. Curly braces { } denote zero or more repetitions; square brackets [ ] denote optional elements. Operator precedence is encoded directly in the grammar hierarchy — no separate precedence table is needed.

### Top-Level Structure

```
<program>       ::= { <top_decl> }
<top_decl>      ::= <workout_decl> | <routine_decl> | <var_decl> |
<func_decl>
```

### Workout Declaration (domain-specific construct)

```
<workout_decl>  ::= "workout" IDENT "{" { <day_decl> } "}"
<day_decl>      ::= "day" IDENT "{" { <set_stmt> } "}"
<set_stmt>      ::= "set" IDENT "reps" INT_LIT "x" INT_LIT
"rest"DURATION_LIT
```

### Routine Declaration

```
<routine_decl>  ::= "routine" IDENT "{" { <stmt> } "}"
```

### Function Declaration

```
<func_decl>     ::= "func" IDENT "(" [ <param_list> ] ")" ":" <type>
<block>
<param_list>    ::= <param> { "," <param> }
<param>         ::= IDENT ":" <type>
```

### Types

```
<type>          ::= "int" | "float" | "bool" | "duration" | "weight"
```

### Statements

```
<block>         ::= "{" { <stmt> } "}"
<stmt>          ::= <var_decl>
                  | <assign_stmt>
                  | <if_stmt>
                  | <while_stmt>
                  | <repeat_stmt>
                  | <load_stmt>
                  | <print_stmt>
                  | <return_stmt>
                  | <set_stmt>
<var_decl>      ::= <type> IDENT "=" <expr>
<assign_stmt>   ::= IDENT "=" <expr>
                  /* Assignment is a statement, not an expression. */
                  /* This prevents = vs == confusion (reliability). */
<if_stmt>       ::= "if" <expr> <block> [ "else" <block> ]
                  /* Dangling-else: else binds to the nearest unmatched if.
*/
<while_stmt>    ::= "while" <expr> <block>
<repeat_stmt>   ::= "repeat" INT_LIT "weeks" <block>
```

```
                /* Domain-specific loop: iterates a fixed number of weeks.
*/
                /* INT_LIT is evaluated once before the loop begins. */
<load_stmt>     ::= "load" IDENT
                /* Domain-specific: loads a named workout into the current
routine. */
<print_stmt>    ::= "print" <expr>
<return_stmt>   ::= "return" <expr>
```

## Expressions (precedence encoded top-to-bottom, lowest to highest)

```
<expr>          ::= <or_expr>
<or_expr>       ::= <and_expr> { "||" <and_expr> }
                /* Left-associative. Short-circuit: right operand not
evaluated if left is false. */
<and_expr>      ::= <eq_expr> { "&&" <eq_expr> }
                /* Left-associative. Short-circuit evaluation. */
<eq_expr>       ::= <rel_expr> { ( "==" | "!=" ) <rel_expr> }
                /* Left-associative. */
<rel_expr>      ::= <add_expr> { ( "<" | "<=" | ">" | ">=" ) <add_expr> }
                /* Left-associative. */
<add_expr>      ::= <mul_expr> { ( "+" | "-" ) <mul_expr> }
                /* Left-associative. */
<mul_expr>      ::= <unary_expr> { ( "*" | "/" ) <unary_expr> }
                /* Left-associative. */
<unary_expr>    ::= "-" <primary>
                | "!" <primary>
                | <primary>
<primary>       ::= INT_LIT
                | FLOAT_LIT
                | DURATION_LIT
                | WEIGHT_LIT
                | BOOL_LIT
                | STRING_LIT
                | IDENT
                | <func_call>
                | "(" <expr> ")"
<func_call>     ::= IDENT "(" [ <arg_list> ] ")"
<arg_list>      ::= <expr> { "," <expr> }
```

## Ambiguity Resolution Summary

- Operator precedence: fully encoded in the six grammar levels above (unary > mul > add > relational > equality > logical-and > logical-or). No separate rule is needed; the grammar is unambiguous for expressions.
- Dangling else: resolved by binding else to the nearest unmatched if — the standard greedy rule, explicitly stated in the grammar comment.
- DURATION_LIT vs. INT_LIT + IDENT: the lexer greedily consumes the numeric suffix (s, min, hr) as part of the token, so 90s is never ambiguous.
- repeat vs. while: repeat takes only an INT_LIT and the keyword weeks; while takes a general boolean expression. They are syntactically distinct and cannot be confused.

# 4.6 Names, Binding, Scope, and Lifetime

## Legal Identifiers

An identifier must match [a-zA-Z_][a-zA-Z0-9_]*. Identifiers are case-sensitive. All keywords (workout, routine, day, set, reps, rest, repeat, weeks, load, func, if, else, while, return, print) and all type names (int, float, bool, duration, weight) are reserved and may not be used as identifiers. The token x is also reserved as a separator.

## Bindings: Compile Time vs. Run Time

- **Compile-time bindings:** the names of workout declarations, routine declarations, and function declarations are bound at compile time. Their types and arities are fixed before execution begins.
- **Run-time bindings:** function parameters and local variables declared inside a routine or function body are bound at run time when the enclosing block is entered. Their values are run-time properties.

## Scoping Rule: Static (Lexical) Scoping

WorkoutLang uses static scoping (Sebesta §5.4). The scope of every name is determined by the textual structure of the program, not by the call sequence at runtime. There are three scope levels:

- **Global scope:** workout, routine, and function declarations. Visible throughout the entire program.
- **Routine/function scope:** parameters and local variables declared inside a routine or function body. Visible only within that block.
- **Block scope:** variables declared inside an if, while, or repeat block. Visible only within that block and any nested blocks.

**Why not dynamic scoping?**

Under dynamic scoping, a function call could accidentally capture variables from the calling routine. For example, if a routine declares int volume and calls a function that also uses volume, dynamic scoping would let the function silently read the routine's variable instead of requiring an explicit parameter. This would make programs extremely hard to read — violating the primary design goal stated in §4.1. Static scoping makes every name resolution traceable to its textual declaration, which is essential for a language used by non-programmers.

## Lifetime of Variables

| Variable Kind | Lifetime | Sebesta Reference |
|---|---|---|
| workout / routine / func names | Static | §5.4.3 — exist for entire program duration |
| Function parameters | Stack-dynamic | §5.4.2 — created on call, destroyed on return |
| Local variables (routine/func) | Stack-dynamic | §5.4.2 — created on block entry, destroyed on exit |
| Block variables (if/while/repeat) | Stack-dynamic | §5.4.2 — created on block entry, destroyed on exit |

WorkoutLang does not support explicit heap allocation. There is no malloc, new, or pointer type. This is a deliberate design decision: the target users are fitness coaches, not programmers. Heap management would add cognitive overhead without any benefit in this domain.